The Relay: write-up

1. What I built and why

A Python assistant that answers only from the three pinned papers (versions stamp-checked):

- **Understand:** rules, no model call. Which papers are named, and is it a comparison?
- **Retrieve:** a `search_papers` tool over 384-token section-aware chunks (BM25 + bge-small fused, top 4, timeout and one retry). A comparison searches each paper separately: the *branch*.
- **Draft:** one gpt-oss-20b call returns JSON claims, each with a chunk ID and an exact quote.
- **Check:** code verifies every quote word for word; one redraft (the *loop*), else refuse.
- **Send:** the reply is built only from checked claims, with citations from chunk metadata.

Every model call goes through one **gateway**: token buckets that refill continuously like Groq's, a bounded FIFO queue, a 10 s admission deadline, capped retries, and a labelled passages-only reply when the daily budget is gone.

The decision that shaped it most: one model call per answer. Groq allows 8,000 tokens/min and reserves prompt + max_tokens (~2,500) per call, so about 3 answers a minute. A model-based checker would halve that, so the check is code; §8 gives its measured price. **"Fast"** means a reply starts in ~1 s when idle (p50 945 ms, n=18), nobody admitted queues more than 10 s, and anyone we can't serve in time is told at once.

2. The real service and my stand-in

Limits came from tiny calls until a 429 (30/min), large prompts until a 429 (8K tokens/min), 10 calls at once (all succeeded), and the `x-ratelimit-*` headers (1,000 requests/day). Surprises:

- Limits **refill continuously** (one request per 86.4 s).
- Admission counts **prompt + max_tokens**, but you are charged for actual use.
- **A 429 costs no daily request.**
- **JSON mode can fail server-side** inside a 200 stream.
- Tokens/day was never reached (documented 200K, low confidence).

Stand-in: same interface as Groq, calibrated by resampling **70 real calls** (first token p50 655 ms, generation p50 186 ms, 168 output tokens). It 429s in ~65 ms with retry-after and the same headers, streams in pieces, fails at the measured JSON rate, has slow outliers, and slows as calls pile up. **Least like Groq:**

- it is in-process, so it has no network delays (§4);
- its slowdown under load is assumed (Groq barely slowed at 10 at once);
- its answers are synthetic, so quality is "not measured" on its rows. Against real runs of the same scenarios it held up: burst 25.0% succeeded on both; steady 94.1% real vs 88.2% stand-in.

3. Results

Tables 1–3 follow, computed only from `results.csv`. **A number we could not have guessed:** Groq's JSON mode rejected **1.25%** of outputs server-side, always on the same maths-heavy question (Q03). **Quality on 18 self-written questions:**

- refusals 5/6 correct (all near-miss and off-topic; the false premise was refused but not corrected);
- answerable 4/12 by the automatic grader and 3/12 by my hand review, at ~2,100 tokens per answer;
- by hand label, **50% of claims (10/20) say more than their own quote**. The grader matched my verdicts on 13/18.

4. Where the time goes

Q04 (real), first token at 3,518 ms. Working: retrieval 24.5 ms, generation 226 ms (212 tokens, 132 reasoning), quote check 1.9 ms. Waiting: 3,265 ms before the model's first token. Retrying: none. Groq's `usage.queue_time` explains only 368 ms, so **the delay was before the first token and mostly outside Groq's reported queue** (network or front-end; unresolved). Shown by four timestamps per request plus Groq's usage fields. Buffering for the check cost 1.9 ms.

5. When a limit is hit

Per-minute. The gateway estimates the wait from the queue and the token refill rate.

- At most 10 s: the request waits in FIFO order. Over 10 s: the person hears "busy, try again in ~N s" within 20–50 ms. Everyone is delayed at most 10 s; whoever arrives once that 10 s is used up is turned away.
- A 429 is retried after its retry-after, only within the deadline.
- When the limits were halved without notice, the gateway learned it from 429s and headers (70% still succeeded, p95 6.9 s).
- A 25 s deadline served as many people under overload, but at p95 24.1 s vs 10.8 s.

Per-day. No model call, and no fallback model (that would dodge the limit): a **labelled passages-only answer**. Because the limit refills, full answers trickle back (one at +135 s). Nobody gets a weaker *generated* answer.

Retrieval: quality versus cost

End to end (Table 3, real Groq): correct answers hybrid 4, bm25 4, dense 3; answerable questions wrongly refused 0, 2 and 3. Only dense challenged the false premise; bm25 accepted it (see the next section). Retrieval alone found a gold page for 91.7% (hybrid, MRR 0.833, 19 ms), 100% (dense, 0.840, 16 ms) and 91.7% (bm25, 0.708, 0.8 ms). A reranker never helped and cost +600–770 ms.

Choice: hybrid, no rerank. It wrongly refused nothing and costs 19 ms of a ~1 s request. The settings differ by about one question (8.3 points each), so this is a design choice, not a measured win. **Near the limit we don't shrink retrieval:** we only degrade to settings the benchmark has measured, so we stop at the passages-only reply.

How this library will mislead a naive retriever

1. **Comparisons pull one paper.** For Q12, one search returned only H-Net passages in every setting (fix: one search per paper).
2. **Shared vocabulary.** For Q04 (Moshi's *Depth Transformer*), "Transformer" pulled the Attention paper's heads-and-layers table into the top 4, and Moshi's own table puts the Temporal Transformer's 32/32 beside the Depth Transformer's 16/6.
3. **Words that match but mean the opposite.** For Q18 (a false premise about convolutional layers), bm25 matched "convolutional" to a passage comparing other models. The reply, *"The Transformer uses separable convolutional layers…"*, carried a genuine quote, so the

check passed.

4. **Facts lost in extraction.** H-Net's 0.5 threshold lives only in Equation 4, which extraction drops (Q09); merged table rows mismatch models and scores.

6. Retries

Policy: retry a 429 after its retry-after (≤ 2 per request, ≤ 20% of traffic, within the deadline); a longer retry-after means the daily limit, so stop; redraft a failed check once.

Worse: in real run 1, three redrafts (Q02, Q03, Q10) each paid a second full call (~2,500 tokens) and still failed; Q10 returned the identical draft. At 8K tokens/min that is ~21 s of budget, one other person's answer.

My expectation was wrong: on the stand-in, blind 429 retries beat our gateway (21 vs 19 answered, 297 calls vs ~62), because a 429 costs no daily request and takes 60 ms. **Retry what is temporary; don't repeat what is deterministic.**

7. What the person sees

At 3 pm, within ~30 ms:

> *[Limited answer] Today's answering budget is used up, so I can't write a checked answer now. Full answers resume in about N minutes. These are the most relevant passages I found; they may not answer your question:* then three cited passages, e.g. *[hnet p.6] 2.2.1 Chunking Layer: "…"*

8. What I'm unsure about, what I left out, what I learned

Least certain assumption: that a verified quote means a supported claim. It doesn't: **50% of claims overstate their quote**, so users can read confident, really-cited sentences that claim too much. A second model call would halve throughput, so I tested a free local rule against my 20 hand labels: numbers must match and ≥ 60% of the claim's content words must appear in its quote (threshold fixed before scoring). It **caught 9 of the 10 unsupported claims but dropped 3 of the 10 good ones** (0.5 → 7 caught, 2 dropped; 0.7 → 10 caught, 5 dropped). Twenty claims is a small set, so this is measured but not yet deployed.

Deliberately not built:

1. a separate HTTP stand-in server (in-process instead);
2. priority classes and circuit breakers (no priority data; a 429 already says when to retry).

Also cut: the 20b vs 120b comparison.

New to me: I had built a hybrid RAG before, focused more on its retrieval pipeline, but never on things like limits, or on behaviour under specific conditions like overload and idle. This was quite interesting to delve into. Groq also refills its limits continuously, and this alone changes how a system should queue its requests (FIFO, or some other way). I also learnt the calculations around token usage in different scenarios. To simulate load, we use an open-loop load generator, so that the delays users experience under load are not hidden. Another thing I learned was how token usage affects throughput: the prompt size, retrieved passages and maximum output tokens all influence how many requests can be admitted within a minute. I had to think about token budgets not just as an API constraint, but as a resource that directly affects how many users the system can serve.

Your feedback on Vaani

It is quite interesting to see voice agents involved. My previous work with voice agents, STT and TTS was mostly at hackathons, with Gnani API models and Bolna. I noticed Vaani is also releasing its own TTS model native to Indian languages, which is very cool. One thing I found slightly confusing was understanding the technical capabilities and limitations behind some of the performance claims from the homepage alone. I would have liked to see more concrete technical documentation or benchmarks explaining how latency and accuracy are measured. Other than that, the use cases also seem great.

How I worked with the assistant

Real transcript: **[DRIVE LINK]** (unedited except my org ID and email are redacted). **Three prompts that mattered:**

1. **The architecture review prompt:** I asked the assistant to review my proposed pipeline and compare the three options for the number of model calls. This helped me understand the throughput cost of adding separate model calls for understanding and verification. I chose a single model call and kept routing and quote validation in code.
2. **The retrieval review prompt:** I asked the assistant to challenge my retrieval design, including chunk sizes, hybrid search, reranking and evaluation. This exposed the risk of using 512-token chunks when the embedding and reranking models had a 512-wordpiece input limit. I changed the chunking to 384 tokens and evaluated retrieval settings rather than assuming a larger chunk was automatically better.
3. **The support-rule prompt:** I told the assistant not to claim that exact-quote validation proves semantic support, and to measure semantic failures on the benchmark instead. I then hand-reviewed every claim against its quote myself, rather than trusting the automatic grader. This exposed a major weakness: a quote could exist verbatim in a retrieved passage while still failing to support the claim beside it. My review found that 10 of 20 claims overstated their supporting quotes.

One thing it got wrong that I caught: the assistant initially treated the quote-validation mechanism as stronger evidence of answer correctness than it really was. I challenged this by manually checking whether each claim was actually supported by its quoted passage. Quote authenticity and semantic support are different things: a quote can be genuine and still not support the claim. So I report the unsupported-claim rate rather than presenting quote validation as a guarantee of factual correctness.

One decision where I overruled it: the assistant measured how often 512-token chunks were truncated by the embedding model and proposed working from that measurement. I didn't accept a one-off measurement: I required 384-token chunks and a guarantee that no chunk could ever exceed either model's input limit, checked under both tokenizers, with nothing dropped at the end of a section or document. Enforcing that guarantee exposed four bugs in its table-splitting code that were silently losing text, which a single measurement would have missed.

What is distinctly mine: the final architecture, experiments and trade-offs were shaped by my own decisions. I chose to prioritise throughput under a shared free-tier quota, freeze a benchmark before evaluating retrieval, manually inspect generated claims, and preserve raw request-level measurements. I also challenged results that looked correct automatically but did not hold up against the original papers. The assistant helped me explore options and identify weaknesses, but I made the final decisions about what to build, what to measure and what limitations to report.

Results tables (required by the brief)

Table 1: Limits found in the real service (computed from probe-* rows)

Limit	Value found	How found	Confidence
Requests per minute	30 OK, then a 429	probe-rpm: tiny calls back to back until the first 429	high
Tokens per minute	8000 (header); 429 after 3 large calls	x-ratelimit-limit-tokens on every response + probe-tpm	high
Requests per day	1000 (header)	x-ratelimit-limit-requests; remaining drops by 1 per call	high
Tokens per day	not observed (documented 200K)	never reached; no header reports it	low
Requests in flight at once	10 of 10 succeeded at once (626–787 ms)	probe-concurrency	medium (only 10 tried)
What it returns at a limit	HTTP 429 in 60 ms, retry-after 2 s (2 429s seen)	probe rows with status rate_limited	high
Time until it recovers	2,072 ms (TPM), 3,180 ms (RPM) after the first 429	next successful call after the 429	medium (one episode each)

Table 2a: Latency

Scenario	Real or stand-in	Requests sent	Duration (s)	Offered rate (per min)	First token p50 / p95 / p99 (ms)	Total time p50 / p95 / p99 (ms)
burst	real	8	1	8 at once	1,053 / 1,388 / 1,388 (n=2)	1,053 / 1,388 / 1,388 (n=2)
burst	stand-in	12	2	12 at once	1,187 / 1,981 / 1,981 (n=3)	1,187 / 1,981 / 1,981 (n=3)
daily_exhausted	stand-in	17	155	6.6	38 / 2,011 / 2,011 (n=17)	38 / 2,011 / 2,011 (n=17)
overload	stand-in	62	304	12.6	6,531 / 10,762 / 10,762 (n=19)	6,531 / 10,762 / 10,762 (n=19)
overload_naive_retry	stand-in	62	299	12.6	2,656 / 6,372 / 6,678 (n=21)	2,656 / 6,372 / 6,678 (n=21)
rpm_halved	stand-in	20	296	4.1	1,536 / 6,914 / 6,914 (n=14)	1,536 / 6,914 / 6,914 (n=14)
steady	real	17	310	3.3	777 / 7,137 / 7,137 (n=16)	777 / 7,137 / 7,137 (n=16)
steady	stand-in	17	236	4.4	1,577 / 5,076 / 5,076 (n=15)	1,577 / 5,076 / 5,076 (n=15)

Table 2b: Outcomes

Scenario	Real or stand-in	Succeeded %	Rate-limited %	Timed out or given up %	Retries per request	Peak queue length	Tokens per answer	Cost per 1,000 answers
burst	real	25.0%	75.0%	0.0%	0.00	2	1,860	$0.17 at list price ($0 on free tier)
burst	stand-in	25.0%	75.0%	0.0%	0.00	1	1,892	$0.17 at list price ($0 on free tier)
daily_exhausted	stand-in	35.3%	64.7%	0.0%	0.00	1	2,102	$0.20 at list price ($0 on free tier)
overload	stand-in	30.6%	67.7%	0.0%	0.00	1	2,030	$0.20 at list price ($0 on free tier)
overload_naive_retry	stand-in	33.9%	66.1%	0.0%	3.77	0	2,111	$0.20 at list price ($0 on free tier)
rpm_halved	stand-in	70.0%	30.0%	0.0%	0.10	1	2,070	$0.19 at list price ($0 on free tier)
steady	real	94.1%	5.9%	0.0%	0.00	1	2,155	$0.21 at list price ($0 on free tier)
steady	stand-in	88.2%	11.8%	0.0%	0.00	1	2,097	$0.20 at list price ($0 on free tier)

Table 3: Retrieval settings

Setting (piece size, pieces fetched, re-rank)	Questions asked	Correct and cited %	Wrong %	Correctly refused %	Tokens per answer	First token p50 / p95 (ms)	Total time p50 / p95 (ms)
384 tokens, 4 pieces, bm25, re-rank no	18	22.2%	38.9%	27.8%	2,340	770 / 2,620	770 / 2,620
384 tokens, 4 pieces, dense, re-rank no	18	16.7%	33.3%	33.3%	2,382	793 / 2,185	793 / 2,185
384 tokens, 4 pieces, hybrid, re-rank no	18	22.2%	50.0%	27.8%	2,124	945 / 3,518	945 / 3,518